

passgen

Возможности и реализация

Далее перечислены все возможности программы passgen, как они работают и какими средствами реализованы.

1. Генерация пароля

Это основная функция. Пользователь вводит сид (секретную фразу), выбирает сервис, нажимает "Создать" — получает пароль.

Реализация: функция `gen()` в JavaScript. Собирает данные из полей, проверяет корректность, вызывает `hmacGen()`, применяет инверсию, выводит результат.

Ключевые моменты:

- Данные берутся через `document.getElementById(id).value`
- Алфавит формируется из СНК по значениям в `state`
- Если сервис не заполнен — используется "(empty)"
- Сид обязателен — иначе розовая подсветка и ошибка
- Выбран хотя бы один набор символов — иначе ошибка
- После генерации пароль автоматически копируется в буфер
- Новый сервис автоматически добавляется в список

2. HMAC-SHA256

Это сердце программы. Именно здесь происходит преобразование сида и сервиса в пароль.

Реализация: функция `hmacGen(seed, svc, len, alpha)`. Использует встроенный в браузер Web Crypto API.

Алгоритм пошагово:

- Seed преобразуется в байты через `TextEncoder`
- Эти байты становятся ключом HMAC через `crypto.subtle.importKey()`
- Сообщение = сервис + длина + байт счётчика
- `crypto.subtle.sign()` возвращает 32 байта (256 бит)
- Каждый байт превращается в символ из алфавита через `byte % len(alpha)`
- Байты больше или равные `max = 256 - (256 % len(alpha))` отбрасываются
- Если нужно >32 символов — счётчик увеличивается и берётся новый HMAC

Важно: отбрасывание байтов (rejection sampling) нужно для равномерного распределения. Без него некоторые символы были бы немного чаще других.

3. Логин

Генерация короткого из 6 символов логина, который можно запомнить.

Реализация: функция `genLogin()`.

- Используется `crypto.subtle.digest("SHA-256")` от сида + "login" + сервис
- Алфавит: "abcdefghijklmnopqrstuvwxyz23456789"
- Нет путающихся символов: l, o, 0, 1
- Берутся первые шесть байтов хэша, каждый превращается в символ: `bytes[i] % len(alpha)`
- Требуется заполненные поля сида и сервиса

Логин предназначен для повседневного использования, а не для криптостойкости.

4. Оценка надёжности

Визуальная индикация стойкости пароля под полем с паролем.

Реализация: функция `updateStrength(len, sets)`.

- Принимает длину и количество включённых наборов
- Рисует три полосы (`.str-bar`) в контейнере `#strBars`
- Количество заполненных полосок = 1 для слабого, 2 для среднего, 3 для сильного
- Цвета через CSS-классы: `.on.weak` (розовый), `.on.medium` (малиновый), `.on.strong` (сиреневый)
- Текст с эмодзи и рекомендацией в `#strInfo`

Критерии оценки:

- Слабый (1 полоска): длина < 10 или наборов < 3
- Средний (2 полоски): длина ≥ 10 и наборов ≥ 3
- Сильный (3 полоски): длина ≥ 14 и наборов ≥ 4

Переключение между уровнями анимировано через CSS transition: `background .4s ease`.

5. Инверсия

Две дополнительные опции в блоке "Расширенные":

Первая заглавная — делает первый символ пароля заглавной буквой.

Последняя цифра — последний символ заменяется на цифру, вычисленную детерминированно.

Реализация в `gen()` после `hmacGen()`:

- `inv.cap`: `finalPwd = finalPwd[0].toUpperCase() + finalPwd.slice(1)`
- `inv.dig`: отдельный HMAC от сида + `"_lastdig"`
- Берётся байт[0] из подписи, делается `% 10`
- Последний символ заменяется на эту цифру

Обе опции накладываются поверх уже готового пароля, не влияя на основную HMAC-генерацию.

6. Сервисы (список и чипы)

Управление списком сервисов для быстрого выбора.

Константа POPULAR:

- Массив объектов с полями: n (имя), l (буква лого), c (цвет)
- Содержит: google, github, vk, yandex, mail, icloud, dropbox, telegram
- Можно добавить свой сервис просто дописав в массив

Функция renderSvcChips():

- Полностью пересобирает сетку #svcGrid
- Сначала популярные сервисы (которых нет в svcs)
- Потом пользовательские из массива svcs

Функция chip(item, hasLogo, deletable):

- Создаёт один элемент .svc-chip
- Если hasLogo == true — цветной кружок с буквой, иначе серый "?"
- При клике по чипу имя попадает в поле ввода
- При правом клике (oncontextmenu) удаляет сервис из svcs
- Панель раскрывается через toggleSvcPanel() с CSS анимацией

7. Пресеты

Быстрые предустановки параметров через выпадающий список.

Реализация: функция `applyPreset()`.

- Читает значение из `<select id="preset">`
- Варианты: "Свой" (ничего не меняет), "Минимальный", "Стандартный", "Максимальный"
- Каждый пресет меняет `state` и `length.value`
- После применения вызывается `renderChk()` для обновления интерфейса

Настройки пресетов:

- Минимальный: 8 символов, буквы + цифры
- Стандартный: 10 символов, всё кроме спецсимволов
- Максимальный: 16 символов, всё

8. Экспорт и импорт

Сохранение и восстановление настроек в JSON-файл.

Реализация `exportData()`:

- Собирает объект: `{svcs, state, inv, len}`
- Преобразует в JSON через `JSON.stringify` с отступами
- Создаёт Blob и скачивает через временную ссылку
- Имя файла: `passgen_export.json`

Реализация `importData(e)`:

- Читает файл через `FileReader`
- Парсит JSON и восстанавливает поля: `svcs, state, inv, len`
- После восстановления вызывает: `renderSvcChips()`, `renderChk()`, `renderInv()`
- При ошибке JSON показывает `alert("Ошибка импорта")`

Важно: сид никогда не попадает в файл экспорта — это сделано намеренно для безопасности.

9. Сохранение настроек

Автоматическое сохранение и восстановление настроек при запуске.

Реализация `saveState()`:

- Сериализует `state`, `svcs` и `len` в JSON
- Сохраняет в `localStorage` под ключом `"passgen_state"`
- Вызывается после каждого изменения настроек

Реализация `loadState()`:

- Вызывается из `init()` при запуске
- Читает JSON из `localStorage`
- Восстанавливает `state`, `inv`, `svcs`, `len`
- После восстановления перерисовывает интерфейс

Локальное хранилище не очищается при закрытии программы, но сбрасывается при очистке кэша браузера.

10. Анимации

Плавность интерфейса через CSS анимации.

Типы анимаций:

- `fadeUp` — появление карточек со сдвигом вверх (из опакитета + `translateY`)
- `glow` — пульсирующая рамка при ошибке (цвет меняется между `var(--error)` и `var(--pink)`)
- `slideDown/slideUp` — раскрытие панели сервисов и расширенных (через `max-height`)
- `checkPop` — пружинка при переключении кружка (`scale 1 -> 1.2 -> 1`)

11. app.py — сервер

Запускает HTTP-сервер на Python без внешних зависимостей.

- Использует SimpleHTTPRequestHandler из стандартной библиотеки
- Слушает на порту 8765 (автопоиск свободного)
- Логи отключены (log_message пропускает)
- Работает в отдельном потоке (daemon=True)
- Ожидание завершения: threading.Event().wait() — кроссплатформенно
- Поиск браузера: перебирает Chrome, Chromium, Brave, Edge, Firefox в --app режиме
- Если ничего не найдено — фалбэк на webbrowser.open()

Итог

Программа полностью работает в браузере, не требует установки дополнительных библиотек, работает на Windows и Linux. Пароли не хранятся, а вычисляются через HMAC-SHA256 возле пользователя, что обеспечивает безопасность и детерминированность.